

Módulos em Python

Organizando Projetos

Ana Júlia Lima



Desenvolvido para
Young 1

Ctrl Play Santa Mônica
Vila Velha, Brasil
Fevereiro 2026

Sumário

1	O que são módulos?	2
2	Para que servem?	2
3	Importando módulos	2
4	Exemplo com math	2
4.1	Explicação	2
5	Excluindo arquivos e pastas	2
6	Módulos embutidos	3
6.1	dir()	3
6.2	help()	3
7	Criando seu próprio módulo	3
8	Importando seu módulo	3
9	Escrevendo pacotes	3
10	Importando módulos de pacotes	3
10.1	Usando import	3
10.2	Usando from	3
11	Apelidos para módulos	4
12	Erros Comuns com Módulos	4
12.1	1. Esquecer de importar o módulo	4
12.2	2. Escrever o nome do módulo errado	4
12.3	3. Tentar importar um arquivo que não está no lugar certo	5
12.4	4. Confundir módulo com função	5
12.5	5. Esquecer o nome do módulo ao usar import normal	5
12.6	6. Confundir import com from	5
12.7	7. Dar ao arquivo o mesmo nome de um módulo embutido	6
12.8	8. Esquecer que pastas de pacote precisam estar bem organizadas	6
12.9	9. Não usar apelidos quando o nome fica grande demais	6
12.10	Resumo	6
13	Lista de Exercícios — Módulos	7
14	Gabarito — Módulos	8
15	Mini Projeto Final — Sistema de Cadastro Usando Módulos	10
15.1	Objetivo do projeto	10
15.2	Estrutura do projeto	10
15.3	Arquivo menu.py	10
15.4	Arquivo cadastro.py	10
15.5	Arquivo principal.py	11
15.6	Como esse projeto funciona?	12
15.7	Exemplo de execução	12
16	Desafios Resolvidos	13

1 O que são módulos?

Módulos são arquivos Python que contêm código reutilizável. Eles podem ter:

- funções
- variáveis
- classes

2 Para que servem?

Módulos ajudam a:

- organizar o código
- evitar repetição
- dividir projetos grandes
- reutilizar funções

3 Importando módulos

Para usar um módulo usamos o comando `import`.

```
import math
```

4 Exemplo com math

```
import math

print(math.pow(2, 3))    # 8.0
print(math.ceil(4.2))   # 5
print(math.trunc(4.9))  # 4
```

4.1 Explicação

- `pow()` → potência
- `ceil()` → arredonda para cima
- `trunc()` → remove parte decimal

5 Excluindo arquivos e pastas

Podemos usar o módulo `os`.

```
import os

os.remove("arquivo.txt")    # remove arquivo
os.rmdir("pasta")           # remove pasta vazia
```

6 Módulos embutidos

Python possui vários módulos prontos.
Podemos explorá-los com:

6.1 `dir()`

```
import math  
  
print(dir(math))
```

Mostra tudo que existe no módulo.

6.2 `help()`

```
help(math)
```

Mostra documentação do módulo.

7 Criando seu próprio módulo

Você pode criar um arquivo Python.
Exemplo: `meu_modulo.py`

```
def saudacao(nome):  
    return "Olá, " + nome
```

8 Importando seu módulo

```
import meu_modulo  
  
print(meu_modulo.saudacao("Ana"))
```

9 Escrevendo pacotes

Pacotes são pastas que contêm módulos.
Estrutura:

```
meu_pacote/  
    modulo1.py  
    modulo2.py
```

10 Importando módulos de pacotes

10.1 Usando `import`

```
import meu_pacote.modulo1
```

10.2 Usando `from`

```
from meu_pacote.modulo1 import funcao
```

11 Apelidos para módulos

Podemos usar apelidos com `as`.

```
import math as m

print(m.sqrt(16))
# Saída: 4.0
```

Resumo Final

- Módulos organizam o código
- Import usamos para acessar módulos
- Python possui módulos prontos
- Podemos criar nossos próprios módulos
- Pacotes agrupam módulos
- Podemos usar apelidos para facilitar

12 Erros Comuns com Módulos

Ao começar a usar módulos, alguns erros aparecem com frequência. Isso é normal, porque agora o programa passa a depender de mais de um arquivo.

Vamos ver os erros mais comuns.

12.1 1. Esquecer de importar o módulo

Se você tentar usar algo de um módulo sem fazer o `import`, o Python não vai reconhecer.

Exemplo errado:

```
print(math.pow(2, 3))
```

Isso gera erro, porque o módulo `math` não foi importado antes.

Correto:

```
import math

print(math.pow(2, 3))
# Saída: 8.0
```

12.2 2. Escrever o nome do módulo errado

Se o nome do módulo estiver errado, o Python não vai encontrar esse arquivo.

Exemplo errado:

```
import maths
```

Se o módulo correto for `math`, isso dará erro.

Correto:

```
import math
```

12.3 3. Tentar importar um arquivo que não está no lugar certo

Quando criamos nosso próprio módulo, o arquivo precisa estar no mesmo local do programa principal ou em um caminho que o Python consiga encontrar.

Por exemplo, se você tiver:

```
projeto/  
    principal.py  
    funcoes.py
```

Então em `principal.py` você pode escrever:

```
import funcoes
```

Mas se o arquivo `funcoes.py` não estiver ali, o `import` falhará.

12.4 4. Confundir módulo com função

Às vezes o aluno tenta usar o módulo como se ele fosse a própria função.

Exemplo errado:

```
import math  
  
print(math)
```

Isso não chama nenhuma função útil do módulo, apenas mostra a referência ao módulo.

O correto é chamar algo **dentro** dele:

```
import math  
  
print(math.ceil(4.2))  
# Saída: 5
```

12.5 5. Esquecer o nome do módulo ao usar import normal

Quando usamos apenas `import math`, precisamos acessar as funções com `math.` antes.

Errado:

```
import math  
  
print(pow(2, 3))
```

Correto:

```
import math  
  
print(math.pow(2, 3))  
# Saída: 8.0
```

12.6 6. Confundir import com from

Esses dois jeitos funcionam, mas são diferentes.

```
import math  
print(math.ceil(4.2))  
  
e  
  
from math import ceil  
print(ceil(4.2))
```

No primeiro caso, usamos `math.`. No segundo, usamos a função diretamente.

12.7 7. Dar ao arquivo o mesmo nome de um módulo embutido

Esse erro é muito comum.

Imagine que você criou um arquivo chamado:

```
math.py
```

Depois, ao tentar usar:

```
import math
```

O Python pode importar o seu arquivo em vez do módulo oficial.

Isso causa muita confusão.

Dica: evite nomes como:

- `math.py`
- `os.py`
- `random.py`

12.8 8. Esquecer que pastas de pacote precisam estar bem organizadas

Quando usamos pacotes, precisamos prestar atenção no caminho.

Se existe esta estrutura:

```
meu_pacote/  
operacoes.py
```

Então o import precisa respeitar isso:

```
import meu_pacote.operacoes
```

ou

```
from meu_pacote.operacoes import soma
```

12.9 9. Não usar apelidos quando o nome fica grande demais

Não é exatamente um erro, mas pode deixar o código confuso.

```
import meu_pacote.operacoes_matematicas_avancadas
```

Pode ser mais prático usar:

```
import meu_pacote.operacoes_matematicas_avancadas as op
```

12.10 Resumo

- Sempre importe o módulo antes de usá-lo.
- Verifique se o nome está correto.
- Coloque seus arquivos no local certo.
- Não confunda módulo com função.
- Cuidado com nomes de arquivos iguais aos módulos embutidos.
- Em pacotes, respeite o caminho do módulo.

13 Lista de Exercícios — Módulos

Nível 1 — Importando módulos prontos

1. Importe o módulo `math` e use `pow()` para calcular 2^5 .
2. Use `math.ceil()` para arredondar o número 7.2 para cima.
3. Use `math.trunc()` para remover a parte decimal de 9.8.
4. Use `dir()` para mostrar os recursos disponíveis no módulo `math`.

Nível 2 — Módulos próprios

1. Crie um arquivo chamado `utilidades.py` com uma função que exiba uma mensagem de boas-vindas.
2. No arquivo principal, importe esse módulo e chame a função criada.
3. Crie um módulo com uma função que receba dois números e retorne a soma deles.
4. Importe essa função usando `from`.

Nível 3 — Pacotes e apelidos

1. Crie uma pasta chamada `meu_pacote` com um módulo chamado `operacoes.py`.
2. Dentro do módulo, crie uma função chamada `dobro()`.
3. Importe esse módulo usando o comando `import`.
4. Importe a função usando o comando `from`.
5. Importe o módulo `math` com apelido e use esse apelido em um exemplo.

14 Gabarito — Módulos

Nível 1

1.

```
import math

print(math.pow(2, 5))
# Saída: 32.0
```

2.

```
import math

print(math.ceil(7.2))
# Saída: 8
```

3.

```
import math

print(math.trunc(9.8))
# Saída: 9
```

4.

```
import math

print(dir(math))
```

Nível 2

1. Arquivo utilidades.py

```
def boas_vindas():
    print("Seja bem-vindo!")
```

2. Arquivo principal

```
import utilidades

utilidades.boas_vindas()
# Saída: Seja bem-vindo!
```

3. Arquivo calculos.py

```
def soma(a, b):
    return a + b
```

4. Arquivo principal

```
from calculos import soma

print(soma(3, 4))
# Saída: 7
```

Nível 3

1 e 2. Arquivo meu_pacote/operacoes.py

```
def dobro(x):  
    return x * 2
```

3. Importando com import

```
import meu_pacote.operacoes  
  
print(meu_pacote.operacoes.dobro(5))  
# Saída: 10
```

4. Importando com from

```
from meu_pacote.operacoes import dobro  
  
print(dobro(5))  
# Saída: 10
```

5. Apelido

```
import math as m  
  
print(m.ceil(3.4))  
# Saída: 4
```

15 Mini Projeto Final — Sistema de Cadastro Usando Módulos

Agora vamos aplicar o conteúdo de módulos em um projeto mais organizado.

Até aqui, fizemos sistemas em um único arquivo. Agora vamos separar o programa em partes, cada uma com sua responsabilidade.

Isso deixa o projeto:

- mais organizado
- mais fácil de entender
- mais fácil de corrigir
- mais fácil de reutilizar

15.1 Objetivo do projeto

Vamos montar um sistema de cadastro com menu, separado em arquivos diferentes.

O sistema permitirá:

- cadastrar pessoas
- mostrar todos os cadastros
- buscar uma pessoa pelo nome
- sair

15.2 Estrutura do projeto

```
projeto_cadastro/  
  principal.py  
  menu.py  
  cadastro.py
```

15.3 Arquivo menu.py

Esse módulo será responsável apenas por mostrar o menu.

```
def mostrar_menu():  
    print("\n--- MENU ---")  
    print("1 - Cadastrar pessoa")  
    print("2 - Mostrar todos os cadastros")  
    print("3 - Buscar pessoa pelo nome")  
    print("4 - Sair")
```

15.4 Arquivo cadastro.py

Esse módulo ficará responsável pelas funções ligadas aos dados.

```
def cadastrar_pessoa(cadastros):  
    nome = input("Digite o nome: ")  
    idade = int(input("Digite a idade: "))  
    cidade = input("Digite a cidade: ")  
  
    pessoa = {  
        "nome": nome,  
        "idade": idade,  
        "cidade": cidade  
    }
```

```

    cadastros.append(pessoa)
    print("Cadastro realizado com sucesso!")

def mostrar_todos(cadastros):
    if len(cadastros) == 0:
        print("Nenhum cadastro encontrado.")
        return

    print("\n--- TODOS OS CADASTROS ---")

    for pessoa in cadastros:
        print("Nome:", pessoa["nome"])
        print("Idade:", pessoa["idade"])
        print("Cidade:", pessoa["cidade"])
        print("-----")

def buscar_pessoa(cadastros):
    if len(cadastros) == 0:
        print("Nenhum cadastro encontrado.")
        return

    nome_busca = input("Digite o nome da pessoa: ")

    for pessoa in cadastros:
        if pessoa["nome"].lower() == nome_busca.lower():
            print("\n--- CADASTRO ENCONTRADO ---")
            print("Nome:", pessoa["nome"])
            print("Idade:", pessoa["idade"])
            print("Cidade:", pessoa["cidade"])
            return

    print("Pessoa não encontrada.")

```

15.5 Arquivo principal.py

Esse será o arquivo principal do sistema.

```

from menu import mostrar_menu
from cadastro import cadastrar_pessoa, mostrar_todos, buscar_pessoa

def main():
    cadastros = []

    while True:
        mostrar_menu()
        opcao = input("Escolha uma opção: ")

        if opcao == "1":
            cadastrar_pessoa(cadastros)

        elif opcao == "2":
            mostrar_todos(cadastros)

        elif opcao == "3":
            buscar_pessoa(cadastros)

        elif opcao == "4":
            print("Encerrando o programa...")
            break

```

```

        else:
            print("Opção inválida. Tente novamente.")

main()

```

15.6 Como esse projeto funciona?

Cada arquivo tem uma responsabilidade:

- `menu.py` → mostra as opções do sistema
- `cadastro.py` → cuida das funções de cadastro, exibição e busca
- `principal.py` → controla o fluxo geral do programa

Essa separação é justamente a ideia principal do uso de módulos.

15.7 Exemplo de execução

```

--- MENU ---
1 - Cadastrar pessoa
2 - Mostrar todos os cadastros
3 - Buscar pessoa pelo nome
4 - Sair
Escolha uma opção: 1
Digite o nome: Ana
Digite a idade: 18
Digite a cidade: Vila Velha
Cadastro realizado com sucesso!

--- MENU ---
1 - Cadastrar pessoa
2 - Mostrar todos os cadastros
3 - Buscar pessoa pelo nome
4 - Sair
Escolha uma opção: 2

--- TODOS OS CADASTROS ---
Nome: Ana
Idade: 18
Cidade: Vila Velha
-----

```

16 Desafios Resolvidos

Exercício 1

Enunciado:

Faça um programa que leia 5 valores inteiros. Conte quantos destes valores digitados são pares e mostre esta informação.

Resolução:

```
contador_pares = 0

for i in range(5):
    numero = int(input("Digite um número: "))

    if numero % 2 == 0:
        contador_pares += 1

print("Quantidade de números pares:", contador_pares)
```

Exercício 2

Enunciado:

Leia um valor inteiro X. Em seguida apresente os 6 valores ímpares consecutivos a partir de X, um valor por linha, inclusive o X se for o caso.

Resolução:

```
x = int(input("Digite um número: "))

contador = 0

while contador < 6:

    if x % 2 != 0:
        print(x)
        contador += 1

    x += 1
```